Plano de Teste — FoodFind 4.0.0

FATEC Ipiranga · Análise e Desenvolvimento de Sistemas (Noturno) · Laboratório de Engenharia de Software · Prof. Veríssimo
Grupo: Pedro Augusto Dantas Aranda e Vinicius Tofani Siqueira · **Data:** 11/11/2026 · **Versão do sistema:** 4.0.0

Regra deste documento: só está marcado **Aprovado** o que o `npm test` comprovou nesta versão. Casos que dependem de navegador real, câmera, impressora, rede externa ou da imagem Docker oficial estão como **Não executado** e devem ser conferidos na homologação.

Casos automatizados	66 (66 aprovados, 0 reprovados)	**Casos manuais**	15 (não executados)
Testes no npm test	66 casos em 6 suítes, todos passando (o runner conta também o arquivo de apoio: “tests 67, pass 67, fail 0”)		

1. Identificação do escopo

Sistema: FoodFind — web app (PWA) para descobrir restaurantes próximos, ver perfil, cardápio, fotos e avaliações e resgatar descontos por QR Code.

Dentro do escopo
HU1 Cadastro/login e sessão · HU2.1 Busca, filtros, mapa e perfil · HU2.2 Avaliações (cadastrados e OSM) · HU3.2 Campanhas · HU3.3 QR Code · HU3.4 Resgate, vouch

2. Estratégia e tipos de teste

Tipo	Como	Ferramenta
Integração / API (caixa-preta)	Cada arquivo de `test/` sobe o servidor real (`node server.js`) em porta livre, com banco temporário, e chama a API por HTTP com cookies, como um navegador.	node:test (nativo), fetch
Dependências externas simuladas	Overpass e Nominatim substituídos por um servidor local (sem internet); outro cenário derruba o OSM para testar o modo degradado.	http nativo
Segurança	Cabeçalhos, SRI, CSRF, limites de tentativas, hash do token no arquivo do banco, magic bytes, links maliciosos, segredo do código.	node:test
Dados / regressão	Migração de banco da Sprint 3, exclusão em cascata, backup e seed.	node:test + sql.js
Interface (complementar)	Roteiro E2E em Chromium headless com CDN servido localmente (mesmos bytes do npm, SRI conferido pelo navegador). Evidência complementar, não substitui a homologação.	Playwright (fora do repositório)
Homologação manual	Checklist em `docs/roteiro-demonstracao-e-homologacao.md`: dispositivos reais, câmera, impressão, acessibilidade, Docker oficial.	Navegadores e celulares reais
Integração contínua	GitHub Actions: `npm ci` + `npm test` (Node 20) e build Docker com espera do healthcheck a cada push/PR.	GitHub Actions

3. Ambiente e pré-requisitos

Item	Valor
Execução registrada	Linux x86_64, Node.js v22.23.4 (o projeto exige ≥ 20; o CI roda em Node 20), npm 10.9.9, 11/11/2026
Comando	`npm ci && npm test` (= `node --test`)
Dependências	sql.js ^1.14.2, qrcode ^1.5.4 (sem outras)
Variáveis usadas nos testes	PORT=0, FOODFIND_DATA_DIR (pasta temporária), FOODFIND_OVERPASS_URL / FOODFIND_NOMINATIM_URL (simulador), ADMIN_EMAIL/ADMIN_PASSWORD de teste, limites por IP altos (exceto no caso CT-09)
Dados de teste	Criados por cada teste (e-mails únicos); seed com contas `*@foodfind.local` (senha FoodFind@2026)
Homologação	Navegador atualizado (Chrome, Safari, Firefox), celular Android e iPhone, HTTPS público para a câmera, impressora A5/A4, Docker com acesso ao Docker Hub

4. Matriz de casos de teste

4.1 Resumo

ID	Caso de uso	Título	Status
CT-01	HU1.1 Cadastro	Cadastro de cliente com sessão iniciada	**Aprovado**
CT-02	HU1.1 Cadastro	Validações do cadastro	**Aprovado**
CT-03	HU1.1 Cadastro	Conta Restaurante e e-mail reservado	**Aprovado**
CT-04	HU1.2 Login	Login com mensagem genérica	**Aprovado**
CT-05	Segurança	Cookie de sessão seguro	**Aprovado**
CT-06	Segurança	Hash do token no banco	**Aprovado**

CT-07	HU1.2 Login	Rotação de sessão e logout	**Aprovado**
CT-08	Segurança	Bloqueio por e-mail após 5 erros	**Aprovado**
CT-09	Segurança	Limite por IP em login e cadastro	**Aprovado**
CT-10	Segurança	Logs sem segredos	**Aprovado**
CT-11	HU4.1 Painel	Permissões do painel	**Aprovado**
CT-12	HU4.1 Painel	CRUD do restaurante	**Aprovado**
CT-13	HU4.1 Painel	Campos obrigatórios do restaurante	**Aprovado**
CT-14	Segurança	Links maliciosos no cadastro	**Aprovado**
CT-15	Segurança	Links maliciosos vindos do OSM	**Aprovado**
CT-16	HU2.1 Busca	Busca OSM + cadastrados	**Aprovado**
CT-17	HU2.1 Busca	Filtros e ordenação	**Aprovado**
CT-18	HU2.1 Busca	OSM fora do ar	**Aprovado**
CT-19	HU2.1 Perfil	Perfil de lugar do OSM por id	**Aprovado**
CT-20	Reivindicação	Pedido, pendência e aprovação	**Aprovado**
CT-21	Reivindicação	Recusa e cancelamento	**Aprovado**
CT-22	Conta	Redefinição de senha pelo administrador	**Aprovado**
CT-23	HU2.2 Avaliações	Permissões para avaliar	**Aprovado**
CT-24	HU2.2 Avaliações	Validação de nota e comentário	**Aprovado**
CT-25	HU2.2 Avaliações	Uma por cliente, média e distribuição	**Aprovado**
CT-26	HU2.2 Avaliações	Ver, editar e excluir a própria avaliação	**Aprovado**
CT-27	HU2.2 Avaliações	Paginação	**Aprovado**
CT-28	HU2.2 Avaliações	Avaliação de restaurante do OSM	**Aprovado**
CT-29	HU2.2 Avaliações	Cascata ao excluir restaurante FoodFind	**Aprovado**
CT-30	HU3.2 Campanhas	Validações da campanha	**Aprovado**
CT-31	HU3.2 Campanhas	Somente o dono gerencia	**Aprovado**
CT-32	HU3.2 Campanhas	Fuso America/Sao_Paulo	**Aprovado**
CT-33	HU3.2 Campanhas	Listar, editar, pausar e estados	**Aprovado**
CT-34	HU3.3 QR Code	QR em SVG só para o dono	**Aprovado**
CT-35	HU3.3 QR Code	Código secreto nunca público	**Aprovado**
CT-36	HU3.2 Campanhas	Lista pública de campanhas	**Aprovado**
CT-37	HU3.4 Resgate	Erros do resgate	**Aprovado**
CT-38	HU3.4 Resgate	Resgate com comprovante	**Aprovado**
CT-39	Segurança	Anti força bruta no resgate	**Aprovado**
CT-40	HU3.4 Voucher	Validação do voucher pelo dono	**Aprovado**
CT-41	HU3.4 Voucher	Voucher expirado	**Aprovado**
CT-42	HU3.4 Benefícios	Histórico "Meus benefícios"	**Aprovado**
CT-43	Dados	Exclusão em cascata	**Aprovado**
CT-44	LGPD	Editar nome	**Aprovado**
CT-45	LGPD	Trocar senha	**Aprovado**
CT-46	LGPD	Sair de todos os dispositivos	**Aprovado**
CT-47	LGPD	Exportar meus dados	**Aprovado**
CT-48	LGPD	Excluir conta de cliente	**Aprovado**
CT-49	LGPD	Excluir conta de dono	**Aprovado**
CT-50	LGPD	Conta do administrador protegida	**Aprovado**
CT-51	Segurança	Cabeçalhos de segurança	**Aprovado**
CT-52	Segurança	SRI e ausência de inline	**Aprovado**
CT-53	Segurança	Proteção CSRF	**Aprovado**
CT-54	Segurança	Upload inválido	**Aprovado**
CT-55	Segurança	Fotos do dono e órfãs	**Aprovado**
CT-56	Segurança	Geocode protegido	**Aprovado**
CT-57	Segurança	Tiles protegidos	**Aprovado**
CT-58	Segurança	/api/map-check restrito	**Aprovado**
CT-59	Infra	Arquivos estáticos	**Aprovado**
CT-60	PWA	Manifest, ícones e service worker	**Aprovado**
CT-61	Infra	Versão única	**Aprovado**
CT-62	Infra	Robustez a erros	**Aprovado**
CT-63	Dados	Migração de banco da Sprint 3	**Aprovado**

CT-64	Infra	FOODFIND_DATA_DIR	**Aprovado**
CT-65	Infra	Backup	**Aprovado**
CT-66	Infra	Seed de demonstração	**Aprovado**
CT-67	HU2.1 Mapa	Permissão de localização com consentimento (navegador real)	**Não executado**
CT-68	HU2.1 Mapa	Mapa com tiles reais, pins agrupados, popup com nota e "Buscar nesta área"	**Não executado**
CT-69	HU2.1 Busca	Busca por endereço/bairro na interface (Nominatim real)	**Não executado**
CT-70	HU2.1 Perfil	Perfil com fotos, cardápio, horários e botões de contato no navegador	**Não executado**
CT-71	HU2.2 Avaliações	Formulário de avaliação na interface (estrelas, edição preenchida, exclusão)	**Não executado**
CT-72	HU3.3 QR Code	Impressão do cartaz A5 em impressora real	**Não executado**
CT-73	HU3.4 Resgate	Leitura do QR com a câmera no Chrome Android (BarcodeDetector)	**Não executado**
CT-74	HU3.4 Resgate	Leitura do QR no Safari iPhone e Firefox (jsQR)	**Não executado**
CT-75	HU3.4 Resgate	Câmera nativa do celular abre /?resgatar= e conclui após login	**Não executado**
CT-76	Acessibilidade	Navegação por teclado e leitor de tela (NVDA/VoiceOver)	**Não executado**
CT-77	Responsividade	Layout em celular real de 360 px	**Não executado**
CT-78	PWA	Instalação do app no Android/iOS e guia de APK	**Não executado**
CT-79	Infra	docker compose up --build com a imagem oficial node:20-alpine	**Não executado**
CT-80	Infra	GitHub Actions no repositório remoto	**Não executado**
CT-81	Segurança	Console do navegador sem violações de CSP com CDN e tiles reais	**Não executado**

4.2 Casos automatizados (detalhe)

CT-01 · Cadastro de cliente com sessão iniciada	**Aprovado**
Caso de uso	HU1.1 Cadastro
Objetivo	Garantir que um visitante cria conta Cliente e já fica logado
Pré-condições	Servidor no ar; e-mail ainda não cadastrado
Dados de entrada	nome, e-mail novo, senha "senha-forte-123", role cliente, acceptTerms
Passos	1. POST /api/auth/register 2. GET /api/auth/me
Resultado esperado	201 com usuário (sem hash da senha); /me retorna o usuário
Critério de aceitação	Conta criada e sessão ativa
Evidência (npm test)	HU1.1 cadastro: cria conta Cliente e já inicia a sessão

CT-02 · Validações do cadastro	**Aprovado**
Caso de uso	HU1.1 Cadastro
Objetivo	Recusar dados inválidos
Pré-condições	Servidor no ar
Dados de entrada	nome vazio; e-mail inválido; senha de 7 caracteres; sem aceite; e-mail duplicado (maiúsculas)
Passos	Enviar cada combinação para /api/auth/register
Resultado esperado	400 (campos, weak_password, terms_required) e 409 para duplicado
Critério de aceitação	Nenhuma conta inválida criada
Evidência (npm test)	HU1.1 cadastro: valida campos, senha mínima de 8, termos e e-mail duplicado

CT-03 · Conta Restaurante e e-mail reservado	**Aprovado**
Caso de uso	HU1.1 Cadastro
Objetivo	Criar conta do tipo Restaurante e bloquear o e-mail do administrador
Pré-condições	ADMIN_EMAIL definido
Dados de entrada	role restaurante; e-mail = ADMIN_EMAIL
Passos	Cadastrar dono; tentar cadastrar com o e-mail do admin
Resultado esperado	Dono criado; e-mail do admin → 403 reserved_email
Critério de aceitação	Admin não pode ser "sequestrado"
Evidência (npm test)	HU1.1 cadastro: conta Restaurante e e-mail do administrador reservado

CT-04 · Login com mensagem genérica	**Aprovado**
Caso de uso	HU1.2 Login
Objetivo	Não revelar se o e-mail existe
Pré-condições	Usuário cadastrado
Dados de entrada	senha errada; e-mail inexistente; e-mail em maiúsculas
Passos	POST /api/auth/login nos 3 casos
Resultado esperado	401 com mesma mensagem e code nos dois erros; 200 no correto
Critério de aceitação	Mensagens idênticas
Evidência (npm test)	HU1.2 login: mensagem genérica não revela se o e-mail existe

CT-05 · Cookie de sessão seguro	Aprovado
Caso de uso	Segurança
Objetivo	Verificar flags do cookie
Pré-condições	Usuário cadastrado
Dados de entrada	login simples e com X-Forwarded-Proto: https
Passos	Ler Set-Cookie
Resultado esperado	HttpOnly e SameSite=Lax; Secure só em HTTPS
Critério de aceitação	Flags presentes
Evidência (npm test)	Sessão: cookie HttpOnly e SameSite; Secure quando atrás de proxy HTTPS

CT-06 · Hash do token no banco	Aprovado
Caso de uso	Segurança
Objetivo	Garantir que o banco não guarda o token puro
Pré-condições	Usuário logado
Dados de entrada	token do cookie
Passos	Abrir foodfind.db e procurar o token
Resultado esperado	Só existe token_hash = SHA-256(token); token ausente do arquivo
Critério de aceitação	Token puro não persiste
Evidência (npm test)	Sessão: o banco guarda só o hash do token, nunca o token

CT-07 · Rotação de sessão e logout	Aprovado
Caso de uso	HU1.2 Login
Objetivo	Login gera nova sessão e invalida a anterior
Pré-condições	Usuário logado
Dados de entrada	cookie antigo
Passos	Logar de novo; usar cookie antigo; logout
Resultado esperado	Cookie antigo → usuário null; logout zera o cookie
Critério de aceitação	Sessão anterior inválida
Evidência (npm test)	Sessão: login gira a sessão (a anterior deixa de valer) e logout encerra

CT-08 · Bloqueio por e-mail após 5 erros	Aprovado
Caso de uso	Segurança
Objetivo	Limitar força bruta de senha
Pré-condições	Usuário cadastrado
Dados de entrada	5 senhas erradas, depois a correta
Passos	POST /api/auth/login repetido
Resultado esperado	4×401, 5ª→429 com Retry-After e "aguarde N minutos"; correta também 429; outro e-mail funciona
Critério de aceitação	Bloqueio temporário com mensagem clara
Evidência (npm test)	Limite de tentativas: 5 senhas erradas bloqueiam o e-mail temporariamente

CT-09 · Limite por IP em login e cadastro	Aprovado
Caso de uso	Segurança
Objetivo	Limitar abuso por IP
Pré-condições	Servidor com limites padrão
Dados de entrada	11 cadastros e 21 logins do mesmo IP
Passos	Enviar em sequência
Resultado esperado	Cadastro 11 → 429; login 21 → 429
Critério de aceitação	Limites aplicados
Evidência (npm test)	Limite de tentativas por IP no login e no cadastro (configuração padrão)

CT-10 · Logs sem segredos	Aprovado
Caso de uso	Segurança
Objetivo	Garantir que senha/token não vão para o log
Pré-condições	Servidor no ar
Dados de entrada	senha "senha-super-secreta-987"
Passos	Cadastrar, logar e ler stdout/stderr
Resultado esperado	Nem senha nem token aparecem
Critério de aceitação	Logs limpos
Evidência (npm test)	Logs do servidor não contêm senhas nem tokens

CT-11 · Permissões do painel	Aprovado
Caso de uso	HU4.1 Painel
Objetivo	Só contas Restaurante gerenciam restaurantes
Pré-condições	Visitante e cliente
Dados de entrada	—
Passos	GET/POST /api/my/restaurants
Resultado esperado	401 visitante; 403 cliente
Critério de aceitação	Acesso negado
Evidência (npm test)	HU4.1 painel: só contas Restaurante gerenciam restaurantes

CT-12 · CRUD do restaurante	Aprovado
Caso de uso	HU4.1 Painel
Objetivo	Criar, listar, editar e excluir o próprio restaurante
Pré-condições	Conta Restaurante
Dados de entrada	dados completos (cardápio com preço "42,90")
Passos	POST, GET, PUT, DELETE; outro dono tenta PUT/DELETE
Resultado esperado	Dados persistem e aparecem no perfil público; outro dono → 404; após excluir → 404
Critério de aceitação	Só o dono altera
Evidência (npm test)	HU4.1 painel: cria, lista, edita e exclui o próprio restaurante

CT-13 · Campos obrigatórios do restaurante	Aprovado
Caso de uso	HU4.1 Painel
Objetivo	Exigir nome e localização
Pré-condições	Conta Restaurante
Dados de entrada	nome vazio; lat inválida; 0,0
Passos	POST /api/my/restaurants
Resultado esperado	400 nos três
Critério de aceitação	Cadastro recusado
Evidência (npm test)	HU4.1 validação: nome e localização obrigatórios

CT-14 · Links maliciosos no cadastro	Aprovado
Caso de uso	Segurança
Objetivo	Recusar javascript:, data:, vbscript: e credenciais em URL
Pré-condições	Conta Restaurante
Dados de entrada	website/delivery/menu/instagram maliciosos; "meurestaurante.com.br"
Passos	POST /api/my/restaurants
Resultado esperado	400 para maliciosos; URL válida normalizada para https://
Critério de aceitação	Nenhum href perigoso salvo
Evidência (npm test)	Links maliciosos: javascript: e data: são recusados no cadastro

CT-15 · Links maliciosos vindos do OSM	Aprovado
Caso de uso	Segurança
Objetivo	Sanitizar website/telefone do OpenStreetMap
Pré-condições	OSM simulado com website "javascript:" e "data:"
Dados de entrada	busca na Av. Paulista
Passos	GET /api/restaurants
Resultado esperado	website vazio; telefone "não informado"; nenhum javascript:/data: na resposta
Critério de aceitação	Resposta limpa
Evidência (npm test)	Links maliciosos vindos do OpenStreetMap são removidos antes de ir para o front

CT-16 · Busca OSM + cadastrados	Aprovado
Caso de uso	HU2.1 Busca
Objetivo	Juntar fontes, ordenar por distância, respeitar o raio, buscar no cardápio
Pré-condições	OSM simulado; restaurante cadastrado
Dados de entrada	lat/lng da Paulista; q="temaki especial"
Passos	GET /api/restaurants
Resultado esperado	Lista ordenada por distância, sem itens fora do raio; busca por item do cardápio
Critério de aceitação	Resultados corretos
Evidência (npm test)	HU2.1 busca: junta OSM + cadastrados, ordena por distância e respeita o raio

CT-17 · Filtros e ordenação	Aprovado
Caso de uso	HU2.1 Busca
Objetivo	Validar categoria (bar, padaria, sorveteria, pizzaria), preço, aberto agora, nota mínima, raio e ordenação
Pré-condições	OSM simulado
Dados de entrada	parâmetros category, price, open, minRating, radius, sort
Passos	GET /api/restaurants com cada filtro
Resultado esperado	Cada filtro restringe corretamente; ordenação por nome/nota
Critério de aceitação	Filtros consistentes
Evidência (npm test)	HU2.1 filtros: categorias do OSM (bar, padaria, sorveteria, pizzaria), preço, aberto agora, nota mínima e ordenação

CT-18 · OSM fora do ar	Aprovado
Caso de uso	HU2.1 Busca
Objetivo	Manter a busca funcionando sem o OpenStreetMap
Pré-condições	Overpass inacessível
Dados de entrada	—
Passos	GET /api/restaurants
Resultado esperado	200 com aviso e só os cadastrados
Critério de aceitação	Sem erro para o usuário
Evidência (npm test)	OSM fora do ar: a API responde só com os cadastrados e avisa

CT-19 · Perfil de lugar do OSM por id	Aprovado
Caso de uso	HU2.1 Perfil
Objetivo	Permitir F5/compartilhamento de #/restaurante/node-N
Pré-condições	OSM simulado
Dados de entrada	node-1001; node-999999; ff-999999
Passos	GET /api/restaurants/:ref
Resultado esperado	200 para existente; 404 para inexistentes
Critério de aceitação	Rota estável
Evidência (npm test)	Perfil de lugar do OSM por id (F5 em #/restaurante/node-1001)

CT-20 · Pedido, pendência e aprovação	Aprovado
Caso de uso	Reivindicação
Objetivo	Fluxo pendente → aprovado pelo administrador
Pré-condições	Conta Restaurante; admin
Dados de entrada	osmRef node-1001 + mensagem
Passos	Enviar pedido; tentar duplicar; rival tenta; dono tenta aprovar; admin aprova; dono edita
Resultado esperado	Pendente não aparece no painel; duplicados 409; só admin aprova; após aprovar substitui o OSM na busca
Critério de aceitação	Restaurante só editável após aprovação
Evidência (npm test)	Reivindicação: pedido pendente, sem edição até aprovar; admin aprova e o dono passa a editar

CT-21 · Recusa e cancelamento	Aprovado
Caso de uso	Reivindicação
Objetivo	Recusar com motivo e cancelar pendente
Pré-condições	Pedido pendente
Dados de entrada	motivo do admin
Passos	Admin recusa; dono vê motivo; novo pedido; cancelar; cancelar recusado
Resultado esperado	Motivo visível; cancelar pendente 200; analisado 409
Critério de aceitação	Estados respeitados
Evidência (npm test)	Reivindicação: recusa com motivo e cancelamento de pedido pendente

CT-22 · Redefinição de senha pelo administrador	Aprovado
Caso de uso	Conta
Objetivo	Alternativa à recuperação por e-mail
Pré-condições	Admin logado; usuário com sessão
Dados de entrada	e-mail do usuário
Passos	POST /api/admin/users/reset-password; login com senha antiga e temporária
Resultado esperado	Sessões encerradas; antiga 401; temporária 200; não-admin 401/403
Critério de aceitação	Senha temporária funcional
Evidência (npm test)	Administrador redefine a senha de um usuário (alternativa à recuperação por e-mail)

CT-23 · Permissões para avaliar	Aprovado
Caso de uso	HU2.2 Avaliações
Objetivo	Só clientes avaliam
Pré-condições	Restaurante cadastrado
Dados de entrada	—
Passos	POST como visitante e como dono; GET como dono
Resultado esperado	401; 403 not_client; canReview=false com motivo
Critério de aceitação	Contas Restaurante não avaliam
Evidência (npm test)	HU2.2 permissões: visitante 401, conta Restaurante 403

CT-24 · Validação de nota e comentário	Aprovado
Caso de uso	HU2.2 Avaliações
Objetivo	Nota inteira 1–5; comentário ≤ 600
Pré-condições	Cliente logado
Dados de entrada	notas 0, 6, 2.5, "abc", null; comentário 601 e 600
Passos	POST reviews
Resultado esperado	400 inválidos / comment_too_long; 201 com 600; 404 restaurante inexistente
Critério de aceitação	Regras aplicadas
Evidência (npm test)	HU2.2 validação: nota inteira de 1 a 5 e comentário de até 600 caracteres

CT-25 · Uma por cliente, média e distribuição	Aprovado
Caso de uso	HU2.2 Avaliações
Objetivo	Reenvio atualiza; nota aparece em lista, perfil e painel
Pré-condições	Dois clientes
Dados de entrada	A: 5 → 4; B: 2
Passos	POST, GET reviews, busca, perfil, painel, filtros minRating e sort=rating
Resultado esperado	count 2, média 3, distribuição correta, nomes abreviados, sem e-mail
Critério de aceitação	Média consistente em todas as telas
Evidência (npm test)	HU2.2 uma avaliação por cliente: reenviar atualiza; média, contagem e distribuição

CT-26 · Ver, editar e excluir a própria avaliação	Aprovado
Caso de uso	HU2.2 Avaliações
Objetivo	CRUD da avaliação do usuário (formulário preenchido)
Pré-condições	Cliente logado
Dados de entrada	nota 3 → 5, comentário editado
Passos	GET/PUT/DELETE /reviews/mine; outro cliente tenta excluir
Resultado esperado	Edição reflete; lista traz "mine"; exclusão zera; outro cliente 404
Critério de aceitação	Usuário controla só a sua
Evidência (npm test)	HU2.2 ver, editar (formulário preenchido) e excluir a própria avaliação

CT-27 · Paginação	Aprovado
Caso de uso	HU2.2 Avaliações
Objetivo	Listar da mais recente para a mais antiga
Pré-condições	7 avaliações
Dados de entrada	page=1/2, limit=5
Passos	GET reviews
Resultado esperado	2 páginas; 5 + 2 itens; ordem decrescente
Critério de aceitação	Paginação correta
Evidência (npm test)	HU2.2 paginação da mais recente para a mais antiga

CT-28 · Avaliação de restaurante do OSM	Aprovado
Caso de uso	HU2.2 Avaliações
Objetivo	Avaliar OSM e manter notas ao ser assumido/excluído
Pré-condições	OSM simulado; admin
Dados de entrada	node-1004
Passos	Avaliar; lugar inexistente; reivindicar e aprovar; avaliar pelo ff; excluir perfil
Resultado esperado	Notas seguem o lugar; uma por cliente; continuam após exclusão do perfil
Critério de aceitação	Avaliações antigas valem
Evidência (npm test)	HU2.2 restaurantes do OpenStreetMap podem ser avaliados e mantêm as notas quando assumidos

CT-29 · Cascata ao excluir restaurante FoodFind	Aprovado
Caso de uso	HU2.2 Avaliações
Objetivo	Remover avaliações de restaurante criado no FoodFind
Pré-condições	Restaurante com avaliação
Dados de entrada	—
Passos	Excluir restaurante; consultar avaliação e exportação
Resultado esperado	Avaliação removida
Critério de aceitação	Sem dados órfãos
Evidência (npm test)	Excluir restaurante criado no FoodFind apaga as avaliações dele

CT-30 · Validações da campanha	Aprovado
Caso de uso	HU3.2 Campanhas
Objetivo	Título e benefício obrigatórios, desconto, período, limite
Pré-condições	Dono com restaurante
Dados de entrada	título/descrição vazios; 0 e 101%; tipo inválido; fim ≤ início; datas inválidas; limite 0 e 2,5
Passos	POST /api/my/campaigns
Resultado esperado	400 com code específico; limite opcional aceito; rótulo em R$
Critério de aceitação	Só campanhas válidas
Evidência (npm test)	HU3.2 validações: título, descrição do benefício, desconto, período e limite

CT-31 · Somente o dono gerencia	Aprovado
Caso de uso	HU3.2 Campanhas
Objetivo	Proteger campanhas e QR
Pré-condições	Dois donos e um cliente
Dados de entrada	—
Passos	Cliente e outro dono acessam rotas /api/my/campaigns
Resultado esperado	403/404/401 conforme o caso
Critério de aceitação	Isolamento entre donos
Evidência (npm test)	HU3.2 dono: só o dono cria e gerencia campanhas do próprio restaurante

CT-32 · Fuso America/Sao_Paulo	Aprovado
Caso de uso	HU3.2 Campanhas
Objetivo	Converter horário local de SP para UTC
Pré-condições	Dono
Dados de entrada	início 2030-01-10T19:30
Passos	Criar campanha
Resultado esperado	startsAt 22:30Z; startsAtLocal 19:30; agendada
Critério de aceitação	Horário correto
Evidência (npm test)	HU3.2 datas em America/Sao_Paulo

CT-33 · Listar, editar, pausar e estados	Aprovado
Caso de uso	HU3.2 Campanhas
Objetivo	Gerenciar e calcular status
Pré-condições	Dono
Dados de entrada	campanhas ativa, agendada, pausada, encerrada, esgotada
Passos	GET, status, PUT
Resultado esperado	Status corretos; edição mantém o código
Critério de aceitação	Gestão completa
Evidência (npm test)	HU3.2 listar, editar, pausar/ativar e estados (agendada, ativa, pausada, encerrada, esgotada)

CT-34 · QR em SVG só para o dono	Aprovado
Caso de uso	HU3.3 QR Code
Objetivo	Gerar QR no servidor com {PUBLIC_URL}/?resgatar=CODIGO
Pré-condições	PUBLIC_URL=https://foodfind.example.com
Dados de entrada	—
Passos	GET qr.svg (normal e download); novo código
Resultado esperado	image/svg+xml, no-store, attachment; código antigo inválido após renovar
Critério de aceitação	QR correto e protegido
Evidência (npm test)	HU3.3 QR Code: SVG gerado no servidor com {PUBLIC_URL}/?resgatar=CODIGO, só para o dono

CT-35 · Código secreto nunca público	Aprovado
Caso de uso	HU3.3 QR Code
Objetivo	Comprovar que o código não aparece em rotas/telas públicas
Pré-condições	Campanha ativa
Dados de entrada	código, minúsculas e formato com hífens
Passos	Percorrer páginas e APIs públicas como visitante e cliente; histórico e exportação
Resultado esperado	Nenhuma ocorrência; lista pública só com campos permitidos
Critério de aceitação	Segredo preservado
Evidência (npm test)	Segredo: o código da campanha nunca aparece em rotas ou telas públicas

CT-36 · Lista pública de campanhas	Aprovado
Caso de uso	HU3.2 Campanhas
Objetivo	Mostrar só vigentes e indicar vagas
Pré-condições	Campanhas vigente, futura, pausada e lotada
Dados de entrada	—
Passos	GET /api/restaurants/:ref/campaigns
Resultado esperado	Vigente e Lotada (hasSlots=false)
Critério de aceitação	Lista correta
Evidência (npm test)	Lista pública: só campanhas vigentes e não pausadas; indica se há vagas

CT-37 · Erros do resgate	Aprovado
Caso de uso	HU3.4 Resgate
Objetivo	Status, code e motivo distintos para cada caso
Pré-condições	Campanhas em todos os estados
Dados de entrada	sem login, dono, vazio, inválido, futura, encerrada, pausada, esgotada, repetida
Passos	POST /api/campaigns/redeem
Resultado esperado	401, 403, 400, 404, 409 not_started, 410, 423, 409 sold_out, 409 already_redeemed; 9 mensagens diferentes
Critério de aceitação	Usuário entende o motivo
Evidência (npm test)	HU3.4 resgate: todos os erros com status, code e motivo distintos

CT-38 · Resgate com comprovante	Aprovado
Caso de uso	HU3.4 Resgate
Objetivo	Aceitar link do QR e formatos; emitir voucher
Pré-condições	Campanha ativa
Dados de entrada	link completo; código em minúsculas com hífens
Passos	POST redeem
Resultado esperado	201 com restaurante, benefício, validade, FF-000000 e código de conferência
Critério de aceitação	Comprovante completo
Evidência (npm test)	HU3.4 resgate: aceita o link do QR, letras minúsculas e hífens; devolve comprovante

CT-39 · Anti força bruta no resgate	Aprovado
Caso de uso	Segurança
Objetivo	Bloquear após 10 códigos inválidos
Pré-condições	Cliente
Dados de entrada	11 códigos errados
Passos	POST redeem
Resultado esperado	10×404, depois 429
Critério de aceitação	Bloqueio aplicado
Evidência (npm test)	Resgate: muitos códigos inválidos bloqueiam temporariamente (anti força bruta)

CT-40 · Validação do voucher pelo dono	Aprovado
Caso de uso	HU3.4 Voucher
Objetivo	Validar por número + código de conferência, uma vez
Pré-condições	Voucher emitido
Dados de entrada	número FF-…, código errado e correto
Passos	GET e POST /api/my/vouchers
Resultado esperado	Dono vê cliente abreviado e não vê o código; errado 400; correto 200; repetido 409
Critério de aceitação	Uso único
Evidência (npm test)	Voucher: dono valida pelo número + código de conferência, uma única vez

CT-41 · Voucher expirado	Aprovado
Caso de uso	HU3.4 Voucher
Objetivo	Não validar após o fim da campanha
Pré-condições	Voucher de campanha encerrada
Dados de entrada	—
Passos	POST use
Resultado esperado	410 voucher_expired
Critério de aceitação	Validade respeitada
Evidência (npm test)	Voucher expirado não é validado

CT-42 · Histórico "Meus benefícios"	Aprovado
Caso de uso	HU3.4 Benefícios
Objetivo	Listar vouchers com status
Pré-condições	Cliente com 2 resgates
Dados de entrada	—
Passos	GET /api/me/redemptions
Resultado esperado	Lista com Emitido/Usado; dono 403; visitante 401
Critério de aceitação	Histórico correto
Evidência (npm test)	Histórico "Meus benefícios": lista os vouchers do cliente com status

CT-43 · Exclusão em cascata	Aprovado
Caso de uso	Dados
Objetivo	Campanha → resgates; restaurante → campanhas
Pré-condições	Resgates existentes
Dados de entrada	—
Passos	Excluir campanha e restaurante
Resultado esperado	Resgates e campanhas removidos; código inválido
Critério de aceitação	Sem órfãos
Evidência (npm test)	Exclusão em cascata: excluir campanha apaga os resgates; excluir restaurante apaga campanhas

CT-44 · Editar nome	Aprovado
Caso de uso	LGPD
Objetivo	Usuário corrige seus dados
Pré-condições	Usuário logado
Dados de entrada	nome vazio e válido
Passos	PUT /api/me
Resultado esperado	401 sem login; 400 vazio; 200
Critério de aceitação	Nome atualizado
Evidência (npm test)	Editar nome

CT-45 · Trocar senha	Aprovado
Caso de uso	LGPD
Objetivo	Trocar senha e encerrar outras sessões
Pré-condições	Usuário com 2 sessões
Dados de entrada	senha atual errada; nova curta; nova válida
Passos	POST /api/me/password
Resultado esperado	Erros específicos; sessão atual mantida, outra encerrada
Critério de aceitação	Senha trocada com segurança
Evidência (npm test)	Trocar senha: exige a atual, mínimo de 8 e encerra as outras sessões

CT-46 · Sair de todos os dispositivos	Aprovado
Caso de uso	LGPD
Objetivo	Encerrar todas as sessões
Pré-condições	Usuário com 2 sessões
Dados de entrada	—
Passos	POST /api/me/logout-all
Resultado esperado	Ambas encerradas
Critério de aceitação	Nenhuma sessão ativa
Evidência (npm test)	Sair de todos os dispositivos

CT-47 · Exportar meus dados	Aprovado
Caso de uso	LGPD
Objetivo	Portabilidade em JSON
Pré-condições	Cliente com avaliação e resgate; dono
Dados de entrada	—
Passos	GET /api/me/export
Resultado esperado	Arquivo JSON com conta, avaliações, benefícios (dono: restaurantes/campanhas); sem senha/token
Critério de aceitação	Dados completos e seguros
Evidência (npm test)	Exportar meus dados (JSON, sem senha nem token)

CT-48 · Excluir conta de cliente	Aprovado
Caso de uso	LGPD
Objetivo	Apagar sessões, avaliações e resgates
Pré-condições	Cliente com dados
Dados de entrada	senha errada e correta
Passos	DELETE /api/me
Resultado esperado	Errada recusa; correta apaga tudo e libera a vaga da campanha
Critério de aceitação	Exclusão completa
Evidência (npm test)	Excluir conta de cliente: confirma com a senha e apaga sessões, avaliações e resgates

CT-49 · Excluir conta de dono	Aprovado
Caso de uso	LGPD
Objetivo	Apagar restaurantes, campanhas, resgates e fotos
Pré-condições	Dono com foto, campanha e resgate
Dados de entrada	—
Passos	DELETE /api/me
Resultado esperado	Tudo removido, inclusive o arquivo da foto
Critério de aceitação	Exclusão completa
Evidência (npm test)	Excluir conta de dono: apaga restaurantes, campanhas, resgates e fotos do disco

CT-50 · Conta do administrador protegida	Aprovado
Caso de uso	LGPD
Objetivo	Impedir que o admin se exclua
Pré-condições	Admin logado
Dados de entrada	—
Passos	DELETE /api/me
Resultado esperado	403
Critério de aceitação	Admin preservado
Evidência (npm test)	A conta do administrador não pode ser excluída pelo app

CT-51 · Cabeçalhos de segurança	Aprovado
Caso de uso	Segurança
Objetivo	CSP sem unsafe-inline, nosniff, Referrer, X-Frame, Permissions, HSTS
Pré-condições	Servidor no ar
Dados de entrada	—
Passos	GET páginas e API
Resultado esperado	Todos os cabeçalhos presentes
Critério de aceitação	Política aplicada
Evidência (npm test)	Cabeçalhos de segurança em páginas e na API

CT-52 · SRI e ausência de inline	Aprovado
Caso de uso	Segurança
Objetivo	Scripts/estilos externos com integrity; sem handlers inline
Pré-condições	index.html e app.js
Dados de entrada	—
Passos	Inspecionar HTML/JS servidos
Resultado esperado	integrity sha384 + crossorigin; sem onclick; sem Google Fonts; jsQR com SRI; sem api.qrserver.com
Critério de aceitação	Front compatível com a CSP
Evidência (npm test)	SRI: todo script e estilo externo do index.html tem integrity e crossorigin

CT-53 · Proteção CSRF	Aprovado
Caso de uso	Segurança
Objetivo	Recusar alterações de outra origem
Pré-condições	Usuário logado
Dados de entrada	Origin externo; Sec-Fetch-Site cross-site; form-urlencoded
Passos	POST/PUT
Resultado esperado	403 bad_origin; 415; mesma origem 200
Critério de aceitação	CSRF bloqueado
Evidência (npm test)	CSRF: POST de outra origem é recusado; JSON obrigatório

CT-54 · Upload inválido	Aprovado
Caso de uso	Segurança
Objetivo	Conferir magic bytes, tipo e tamanho
Pré-condições	Conta Restaurante
Dados de entrada	PNG/JPG/WEBP reais; HTML disfarçado; GIF; > 5 MB; > 8 MB; cliente
Passos	POST /api/uploads
Resultado esperado	Válidos 201 com extensão pelo conteúdo; demais 400/413/403
Critério de aceitação	Só imagens reais
Evidência (npm test)	Upload: confere o conteúdo real (magic bytes), tipo e tamanho

CT-55 · Fotos do dono e órfãs	Aprovado
Caso de uso	Segurança
Objetivo	Impedir uso de fotos alheias e apagar removidas
Pré-condições	Dois donos
Dados de entrada	—
Passos	Usar foto de outro; remover foto; excluir restaurante
Resultado esperado	Foto alheia ignorada; arquivos removidos do disco
Critério de aceitação	Sem fotos órfãs
Evidência (npm test)	Fotos: só as do próprio dono são aceitas e as removidas são apagadas do disco

CT-56 · Geocode protegido	Aprovado
Caso de uso	Segurança
Objetivo	Origem do site, cache e limite por IP
Pré-condições	Nominatim simulado
Dados de entrada	—
Passos	GET /api/geocode sem/ com origem; repetir; exceder limite
Resultado esperado	403 sem origem; 200; cache; 429 após o limite
Critério de aceitação	Proxy não pode ser abusado
Evidência (npm test)	Geocode protegido: exige origem do próprio site, tem cache e limite por IP

CT-57 · Tiles protegidos	Aprovado
Caso de uso	Segurança
Objetivo	Exigir origem do site
Pré-condições	—
Dados de entrada	—
Passos	GET /tiles sem origem; fora da grade
Resultado esperado	403; 404
Critério de aceitação	Proxy de mapa protegido
Evidência (npm test)	Tiles protegidos: exigem origem do próprio site

CT-58 · /api/map-check restrito	Aprovado
Caso de uso	Segurança
Objetivo	Só admin (ou modo dev)
Pré-condições	Produção/teste
Dados de entrada	—
Passos	GET como visitante e cliente
Resultado esperado	401; 403
Critério de aceitação	Diagnóstico protegido
Evidência (npm test)	/api/map-check: só para o administrador fora do modo de desenvolvimento

CT-59 · Arquivos estáticos	Aprovado
Caso de uso	Infra
Objetivo	gzip, ETag/304, 404, sem path traversal
Pré-condições	Servidor no ar
Dados de entrada	caminhos com ../ codificados
Passos	GET
Resultado esperado	gzip válido; 304; 404; nenhum arquivo fora de public/
Critério de aceitação	Servir com segurança
Evidência (npm test)	Arquivos estáticos: gzip, ETag/304, 404 para arquivo inexistente e sem path traversal

CT-60 · Manifest, ícones e service worker	Aprovado
Caso de uso	PWA
Objetivo	App instalável sem cachear a API
Pré-condições	—
Dados de entrada	—
Passos	GET manifest, ícones, sw.js
Resultado esperado	pt-BR, standalone, ícones 200, SW ignora /api/
Critério de aceitação	PWA válido
Evidência (npm test)	PWA: manifest, ícones e service worker que não guarda a API

CT-61 · Versão única	Aprovado
Caso de uso	Infra
Objetivo	Mesma versão em package, README, servidor e front
Pré-condições	—
Dados de entrada	—
Passos	Comparar arquivos e /api/health
Resultado esperado	4.0.0 em todos
Critério de aceitação	Versões consistentes
Evidência (npm test)	Versão: package.json, README, servidor e front-end usam a mesma versão

CT-62 · Robustez a erros	Aprovado
Caso de uso	Infra
Objetivo	Requisições inválidas não derrubam o servidor
Pré-condições	—
Dados de entrada	JSON quebrado; URL malformada; método inexistente
Passos	Enviar e depois /api/health
Resultado esperado	400/404 e servidor no ar
Critério de aceitação	Processo não cai
Evidência (npm test)	Erros não derrubam o servidor: JSON inválido, URL estranha e rota inexistente

CT-63 · Migração de banco da Sprint 3	Aprovado
Caso de uso	Dados
Objetivo	Converter banco antigo sem perder dados
Pré-condições	Banco no formato antigo
Dados de entrada	usuários, sessão com token puro, restaurante, avaliação, campanha
Passos	Subir o servidor sobre o banco antigo
Resultado esperado	Sessão antiga descartada; login, avaliação e código antigo funcionam; user_version 4 e FKs CASCADE; reabrir sem mudanças
Critério de aceitação	Compatível com bancos existentes
Evidência (npm test)	Migração: banco da Sprint 3 (texto puro nos tokens) é convertido sem perder dados

CT-64 · FOODFIND_DATA_DIR	Aprovado
Caso de uso	Infra
Objetivo	Dados na pasta configurada
Pré-condições	—
Dados de entrada	—
Passos	Verificar arquivos
Resultado esperado	foodfind.db e uploads/ na pasta indicada
Critério de aceitação	Configuração respeitada
Evidência (npm test)	FOODFIND_DATA_DIR: banco e fotos ficam na pasta indicada

CT-65 · Backup	Aprovado
Caso de uso	Infra
Objetivo	Copiar dados com data e hora
Pré-condições	Banco existente
Dados de entrada	FOODFIND_BACKUP_DIR fora e dentro da pasta de dados
Passos	npm run backup (3x)
Resultado esperado	Pastas foodfind-AAAAMMDD-HHMMSS sem recursão
Critério de aceitação	Backup íntegro
Evidência (npm test)	Backup: npm run backup copia a pasta de dados com data e hora

CT-66 · Seed de demonstração	Aprovado
Caso de uso	Infra
Objetivo	Criar dados de demo e proteger contra uso com servidor no ar
Pré-condições	Pasta vazia
Dados de entrada	—
Passos	npm run seed; subir; tentar seed com servidor no ar; parar
Resultado esperado	Contas/campanhas/avaliações criadas; seed recusa com código 2; trava liberada no SIGTERM
Critério de aceitação	Demo pronta e segura
Evidência (npm test)	Seed: cria contas, restaurantes, campanhas e avaliações de demonstração

4.3 Casos de homologação manual

ID	Caso de uso	Título	Pré-condições	Resultado esperado / critério	Status
CT-67	HU2.1 Mapa	Permissão de localização com consentimento (navegador real)	Ambiente de homologação com internet; contas do seed	Aviso antes de pedir; permitir → restaurantes próximos; negar → busca por endereço	Não executado
CT-68	HU2.1 Mapa	Mapa com tiles reais, pins agrupados, popup com nota e "Buscar nesta área"	Ambiente de homologação com internet; contas do seed	Tiles carregam; agrupamento; resumo; nova busca na área	Não executado
CT-69	HU2.1 Busca	Busca por endereço/bairro na interface (Nominatim real)	Ambiente de homologação com internet; contas do seed	Resultados reais e recentralização do mapa	Não executado
CT-70	HU2.1 Perfil	Perfil com fotos, cardápio, horários e botões de contato no navegador	Ambiente de homologação com internet; contas do seed	Galeria, horário do dia destacado, links abrem em nova aba	Não executado
CT-71	HU2.2 Avaliações	Formulário de avaliação na interface (estrelas, edição preenchida, exclusão)	Ambiente de homologação com internet; contas do seed	Fluxo completo pela tela	Não executado
CT-72	HU3.3 QR Code	Impressão do cartaz A5 em impressora real	Ambiente de homologação com internet; contas do seed	Cartaz em uma página A5, QR legível	Não executado
CT-73	HU3.4 Resgate	Leitura do QR com a câmera no Chrome Android (BarcodeDetector)	Ambiente de homologação com internet; contas do seed	Lê o QR e emite o voucher	Não executado
CT-74	HU3.4 Resgate	Leitura do QR no Safari iPhone e Firefox (jsQR)	Ambiente de homologação com internet; contas do seed	Lê o QR e emite o voucher	Não executado
CT-75	HU3.4 Resgate	Câmera nativa do celular abre /?resgatar= e conclui após login	Ambiente de homologação com internet; contas do seed	Resgate automático após login; URL limpa	Não executado
CT-76	Acessibilidade	Navegação por teclado e leitor de tela (NVDA/VoiceOver)	Ambiente de homologação com internet; contas do seed	Tab/Enter/Espaço/ESC; foco preso nos modais; mensagens anunciadas	Não executado
CT-77	Responsividade	Layout em celular real de 360 px	Ambiente de homologação com internet; contas do seed	Sem rolagem horizontal; menus utilizáveis	Não executado
CT-78	PWA	Instalação do app no Android/iOS e guia de APK	Ambiente de homologação com internet; contas do seed	App instalado abre em tela cheia	Não executado
CT-79	Infra	docker compose up --build com a imagem oficial node:20-alpine	Ambiente de homologação com internet; contas do seed	Container (healthy), não-root, dados persistem após recriação	Não executado
CT-80	Infra	GitHub Actions no repositório remoto	Ambiente de homologação com internet; contas do seed	Workflow verde em push/PR	Não executado
CT-81	Segurança	Console do navegador sem violações de CSP com CDN e tiles reais	Ambiente de homologação com internet; contas do seed	Nenhuma violação	Não executado

Passos detalhados: `docs/roteiro-demonstracao-e-homologacao.md`, seções 2 e 3.

5. Relatório de defeitos e evidências

5.1 Defeitos encontrados e corrigidos

ID	Severidade	Descrição	Correção	Reteste
D-01	Alta	Código da campanha exposto em GET /api/restaurants/ff-N/campaigns e QR visível a todos no perfil	Lista pública só com campos permitidos; QR apenas em rotas do dono	CT-35, CT-36
D-02	Média	Nota média sempre null para restaurantes cadastrados	Resumo de avaliações por place_ref em busca, perfil, pin e painel	CT-25
D-03	Média	Restaurantes do OSM não podiam ser avaliados (404)	Avaliações por place_ref (ff-N ou node/way/relation-N)	CT-28
D-04	Média	Sem editar/excluir a própria avaliação e sem paginação	Rotas /reviews/mine (GET/PUT/DELETE) e paginação	CT-26, CT-27

D-05	Média	Dono não listava/pausava/editava/excluía campanhas (404)	Rotas /api/my/campaigns completas e tela de gestão	CT-33
D-06	Baixa	Descrição do benefício opcional no servidor	Validação obrigatória (invalid_description)	CT-30
D-07	Média	QR gerado por api.qrserver.com (vazava o código, não funcionava offline, sem link)	QR SVG no servidor com {PUBLIC_URL}/?resgatar=CODIGO; imprimir/baixar	CT-34, CT-52
D-08	Média	Leitura só com BarcodeDetector e só dentro do perfil	Fallback jsQR (SRI) e botão global "Ler QR Code"	CT-52 (+ CT-73/74 manuais)
D-09	Média	Sem voucher, sem nome do restaurante no resgate e sem histórico	Voucher FF-000123 + código de conferência; /api/me/redemptions	CT-38, CT-40, CT-42
D-10	Baixa	Mesma mensagem para não iniciada, expirada e pausada	Status e code distintos por motivo	CT-37
D-11	Alta	Testes testavam cópias das funções, sem subir o servidor	Suíte nova com servidor real e OSM simulado	CT-01 a CT-66
D-12	Alta	node_modules e banco com dados pessoais versionados; versões 2.2; pastas duplicadas	.gitignore, estrutura única, versão 4.0.0	CT-61
D-13	Alta	Login com e-mail inexistente respondia 404 (enumeração de contas)	Mensagem e tempo equivalentes	CT-04
D-14	Alta	Qualquer dono podia usar (e depois apagar) fotos de outro dono	Tabela uploads com dono; cleanPhotos filtra	CT-55
D-15	Média	Fallback de estáticos devolvia index.html para arquivos inexistentes; /api/map-check público	404 para arquivos; map-check só admin/dev	CT-58, CT-59
D-16	Média	Encontrado no teste E2E: foco do modal de login roubava o cursor e a senha ia para o campo de e-mail	Foco síncrono só se o foco não estiver no modal	Smoke E2E (seção 5.3)
D-17	Média	Encontrado no Docker: npm run backup falhava com a pasta de backups dentro da pasta de dados	Cópia item a item, pulando a pasta de backups	CT-65
D-18	Alta	Encontrado no Docker: seed com o servidor no ar era sobrescrito pelo banco em memória	Trava foodfind.lock; seed recusa; SIGTERM libera	CT-66
D-19	Baixa	Revisão: service worker clonava a resposta depois de entregue (body already used)	Clone antes de devolver	CT-60

Defeitos em aberto: **nenhum conhecido**. Limitações (não são defeitos): dependência do OpenStreetMap, câmera exige HTTPS, ausência de envio de e-mail.

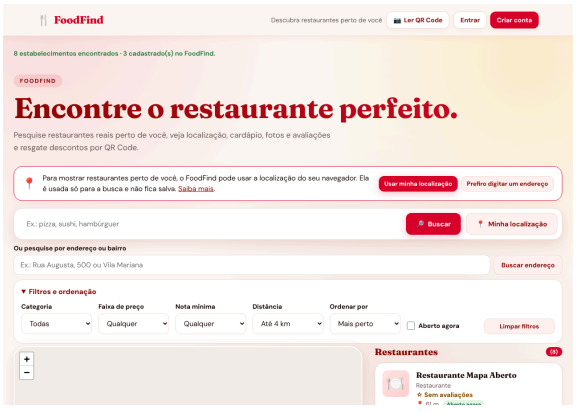
5.2 Saída do npm test (resumo)

```
ℹ tests 67
ℹ suites 6
ℹ pass 67
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ duration_ms 11698.836647
```

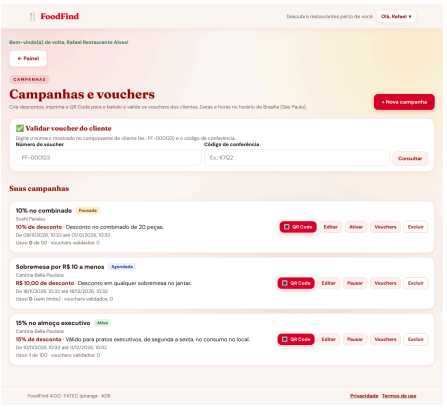
Saída completa por caso: anexo (seção 5.4).

5.3 Evidências complementares de interface

Capturas do roteiro E2E em Chromium headless, com OpenStreetMap simulado e servidores de mapa bloqueados pela rede do ambiente de desenvolvimento (por isso o mapa aparece sem desenho e com o aviso). Não alteram o status dos casos manuais.



Início: aviso de localização, busca, filtros e lista com nota



Gestão de campanhas com status, usos e limite



Comprovante após resgate pelo link do QR (360 px)



Cartaz A5 para impressão com o QR

Verificações registradas nessa execução: sem erros de console nem violações de CSP; SRI de Leaflet/markercluster aceito pelo navegador; 0 px de rolagem horizontal em 360 px; foco preso nos modais e ESC; login/resgate automático pelo link `?resgatar=` com URL limpa; reivindicação aprovada pela tela de administração; exclusão de conta pela interface.

Docker: como o Docker Hub não estava acessível no ambiente, a imagem foi construída sobre uma base local que substitui `node:20-alpine` (mesmo Dockerfile, sem alterações). Resultado: container *healthy*, processo com uid 1000, dados e sessão preservados após `docker compose down` + recriação, backup e seed funcionando. Com a imagem oficial, o caso fica para a homologação (CT-79).

5.4 Anexo — saída completa do npm test

```
✔ test/_helpers.js (41.669271ms)
▶ Conta e LGPD
  ✔ Editar nome (99.230072ms)
  ✔ Trocar senha: exige a atual, mínimo de 8 e encerra as outras sessões (293.668563ms)
  ✔ Sair de todos os dispositivos (80.220182ms)
  ✔ Exportar meus dados (JSON, sem senha nem token) (137.907391ms)
  ✔ Excluir conta de cliente: confirma com a senha e apaga sessões, avaliações e resgates (220.831128ms)
  ✔ Excluir conta de dono: apaga restaurantes, campanhas, resgates e fotos do disco (245.384125ms)
  ✔ A conta do administrador não pode ser excluída pelo app (77.475927ms)
✔ Conta e LGPD (1411.588751ms)
▶ Autenticação e sessão
  ✔ HU1.1 cadastro: cria conta Cliente e já inicia a sessão (70.553372ms)
  ✔ HU1.1 cadastro: valida campos, senha mínima de 8, termos e e-mail duplicado (57.720877ms)
  ✔ HU1.1 cadastro: conta Restaurante e e-mail do administrador reservado (51.268236ms)
  ✔ HU1.2 login: mensagem genérica não revela se o e-mail existe (187.810527ms)
  ✔ Sessão: cookie HttpOnly e SameSite; Secure quando atrás de proxy HTTPS (125.508516ms)
  ✔ Sessão: o banco guarda só o hash do token, nunca o token (94.397402ms)
  ✔ Sessão: login gira a sessão (a anterior deixa de valer) e logout encerra (112.351473ms)
  ✔ Limite de tentativas: 5 senhas erradas bloqueiam o e-mail temporariamente (311.696398ms)
  ✔ Limite de tentativas por IP no login e no cadastro (configuração padrão) (1369.803539ms)
  ✔ Logs do servidor não contêm senhas nem tokens (77.925067ms)
✔ Autenticação e sessão (2638.611349ms)
▶ Campanhas, QR Code, resgate e vouchers
  ✔ HU3.2 validações: título, descrição do benefício, desconto, período e limite (61.940946ms)
  ✔ HU3.2 dono: só o dono cria e gerencia campanhas do próprio restaurante (116.061646ms)
  ✔ HU3.2 datas em America/Sao_Paulo (7.373104ms)
  ✔ HU3.2 listar, editar, pausar/ativar e estados (agendada, ativa, pausada, encerrada, esgotada) (104.103249ms)
  ✔ HU3.3 QR Code: SVG gerado no servidor com {PUBLIC_URL}/?resgatar=CODIGO, só para o dono (75.479956ms)
  ✔ Segredo: o código da campanha nunca aparece em rotas ou telas públicas (151.6944ms)
  ✔ Lista pública: só campanhas vigentes e não pausadas; indica se há vagas (73.98823ms)
  ✔ HU3.4 resgate: todos os erros com status, code e motivo distintos (139.447065ms)
  ✔ HU3.4 resgate: aceita o link do QR, letras minúsculas e hífens; devolve comprovante (97.830508ms)
  ✔ Resgate: muitos códigos inválidos bloqueiam temporariamente (anti força bruta) (66.687372ms)
  ✔ Voucher: dono valida pelo número + código de conferência, uma única vez (119.492718ms)
  ✔ Voucher expirado não é validado (54.037184ms)
  ✔ Histórico "Meus benefícios": lista os vouchers do cliente com status (68.773296ms)
  ✔ Exclusão em cascata: excluir campanha apaga os resgates; excluir restaurante apaga campanhas (110.638943ms)
✔ Campanhas, QR Code, resgate e vouchers (1540.040533ms)
▶ Restaurantes, busca e reivindicação
  ✔ HU4.1 painel: só contas Restaurante gerenciam restaurantes (65.013987ms)
  ✔ HU4.1 painel: cria, lista, edita e exclui o próprio restaurante (162.111791ms)
  ✔ HU4.1 validação: nome e localização obrigatórios (54.836437ms)
  ✔ Links maliciosos: javascript: e data: são recusados no cadastro (66.452159ms)
  ✔ Links maliciosos vindos do OpenStreetMap são removidos antes de ir para o front (47.825038ms)
  ✔ HU2.1 busca: junta OSM + cadastrados, ordena por distância e respeita o raio (53.630692ms)
  ✔ HU2.1 filtros: categorias do OSM (bar, padaria, sorveteria, pizzaria), preço, aberto agora, nota mínima e ordenação (78.057811ms)
  ✔ OSM fora do ar: a API responde só com os cadastrados e avisa (272.641382ms)
  ✔ Perfil de lugar do OSM por id (F5 em #/restaurante/node-1001) (11.968971ms)
  ✔ Reivindicação: pedido pendente, sem edição até aprovar; admin aprova e o dono passa a editar (126.28114ms)
  ✔ Reivindicação: recusa com motivo e cancelamento de pedido pendente (64.690627ms)
  ✔ Administrador redefine a senha de um usuário (alternativa à recuperação por e-mail) (143.283319ms)
✔ Restaurantes, busca e reivindicação (1385.875781ms)
▶ Avaliações
  ✔ HU2.2 permissões: visitante 401, conta Restaurante 403 (18.93602ms)
  ✔ HU2.2 validação: nota inteira de 1 a 5 e comentário de até 600 caracteres (67.553321ms)
  ✔ HU2.2 uma avaliação por cliente: reenviar atualiza; média, contagem e distribuição (160.573499ms)
  ✔ HU2.2 ver, editar (formulário preenchido) e excluir a própria avaliação (117.428207ms)
  ✔ HU2.2 paginação da mais recente para a mais antiga (386.449202ms)
  ✔ HU2.2 restaurantes do OpenStreetMap podem ser avaliados e mantêm as notas quando assumidos (191.128717ms)
  ✔ Excluir restaurante criado no FoodFind apaga as avaliações dele (61.313747ms)
✔ Avaliações (1334.871131ms)
▶ Segurança e infraestrutura
  ✔ Cabeçalhos de segurança em páginas e na API (45.02091ms)
  ✔ SRI: todo script e estilo externo do index.html tem integrity e crossorigin (16.743232ms)
  ✔ CSRF: POST de outra origem é recusado; JSON obrigatório (64.838263ms)
  ✔ Upload: confere o conteúdo real (magic bytes), tipo e tamanho (301.432179ms)
  ✔ Fotos: só as do próprio dono são aceitas e as removidas são apagadas do disco (338.643676ms)
  ✔ Geocode protegido: exige origem do próprio site, tem cache e limite por IP (328.290521ms)
  ✔ Tiles protegidos: exigem origem do próprio site (5.928077ms)
  ✔ /api/map-check: só para o administrador fora do modo de desenvolvimento (47.192026ms)
  ✔ Arquivos estáticos: gzip, ETag/304, 404 para arquivo inexistente e sem path traversal (40.861157ms)
  ✔ PWA: manifest, ícones e service worker que não guarda a API (17.677369ms)
  ✔ Versão: package.json, README, servidor e front-end usam a mesma versão (3.476103ms)
```

```
  ✔ Erros não derrubam o servidor: JSON inválido, URL estranha e rota inexistente (7.093094ms)
  ✔ Migração: banco da Sprint 3 (texto puro nos tokens) é convertido sem perder dados (748.621233ms)
  ✔ FOODFIND_DATA_DIR: banco e fotos ficam na pasta indicada (0.261571ms)
  ✔ Backup: npm run backup copia a pasta de dados com data e hora (131.445226ms)
  ✔ Seed: cria contas, restaurantes, campanhas e avaliações de demonstração (740.049758ms)
✔ Segurança e infraestrutura (3037.934051ms)
ℹ tests 67
ℹ suites 6
ℹ pass 67
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 11698.836647
```